Clothing Assistant Chatbot with Retrieval-Augmented Generation

Final Project Report

Team members :-

Manish S

Madhav V

Neeraj Yadav

Sri Harsha Reddy Sagili

Adarsh Pradhan

1. Define the objectives of the project:

Objective: To build a Clothing Assistant.

A chatbot designed to answer questions from store policy, previous purchases of customers, and suggest outfits based on query given and customer's previous purchases.

Part A: Store policy + Customer help desk

The chatbot will answer questions from store policies like what is the return policy, upcoming discount info, and so on. Additionally the customer can select a previously bought item and ask something like whether I can return or replace the product. Now the agent will check the return or replacement policy of that specific product and reply accordingly.

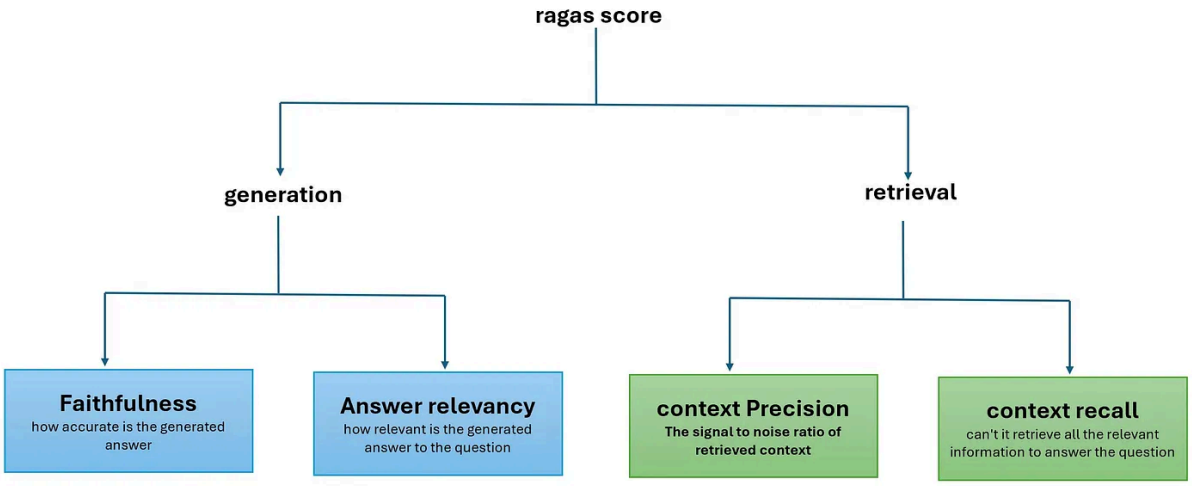
Part B: AI stylist

- The agent will be fed context about the customer preferences (previous purchases, size, etc). Without any query given the agent should concentrate on these preferences and suggest clothes. An additional advanced option could be suggesting clothes based on the skin tone of the customer if shared.
- Initially to get customer preferences and serve well a quiz will be designed.
- Given a query the agent should consider both customer preferences and query given and suggest clothes. (An example query would be 'suggest me an outfit for a graduation party')
- To prevent hallucinations the agent will only suggest products from the company catalogue. Also the agent should stick to the job of answering from company policy or suggesting outfits.
- The agent should have knowledge about fashion like which color shirt matches with which color pants, on which occasions what outfit is appropriate, etc,...

Scoring Metric for Policy Rag:

Retrieval-Augmented Generation Assessment Suite (RAGAS)

RAGAS is a specialized evaluation framework for RAG systems. It goes beyond basic overlap metrics (BLEU, ROUGE) by focusing on how well the generated answer is grounded in retrieved evidence and how relevant / faithful it is.



The main components are:

Faithfulness: Measures how factually correct the generated response is, relative to the information in the retrieved documents.

$$\text{Faithfulness} = \frac{\text{Number of Correct Facts in the Response}}{\text{Total Number of Facts in the Response}}$$

Answer Relevancy: Assesses how well the answer addresses the original query (i.e. relevance of concepts used).

$$\text{Answer Relevancy} = \frac{\text{Number of Relevant Concepts in the Response}}{\text{Total Number of Concepts in the Response}}$$

Context Precision: Among the retrieved passages, how many are actually relevant to the query (i.e. proportion of useful retrieval).

$$\text{Context Precision} = \frac{\text{Number of Relevant Sentences Retrieved}}{\text{Total Number of Sentences Retrieved}}$$

Context Recall: Of all the potentially relevant sections, how many relevant ones were successfully retrieved.

$$\text{Context Recall} = \frac{\text{Number of Relevant Sentences Retrieved}}{\text{Total Number of Relevant Sentences Available}}$$

Composite Score:

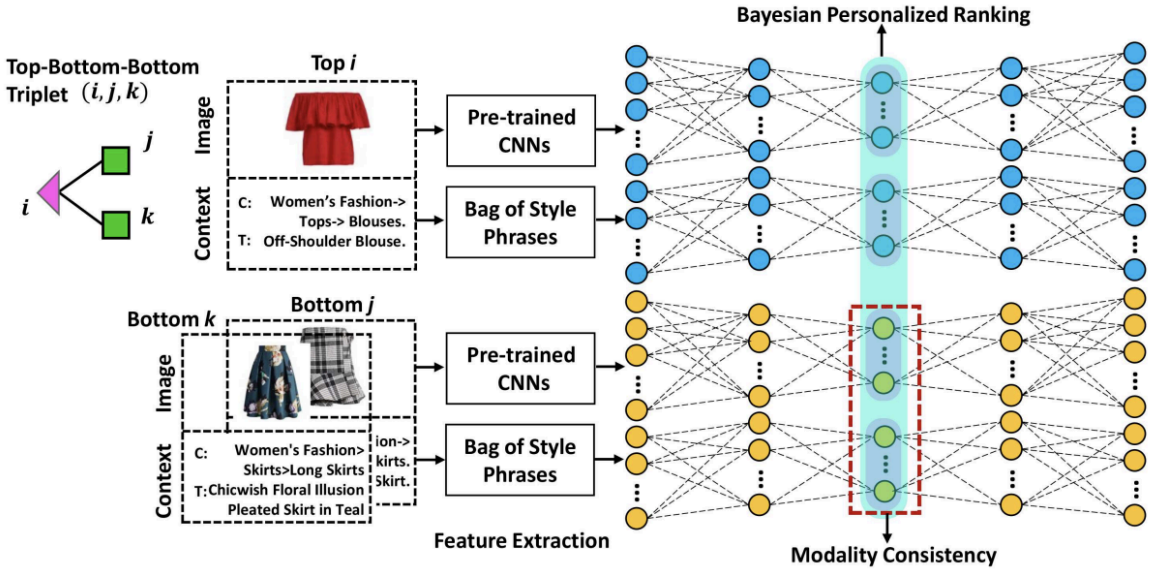
$$\text{RAGAS} = \frac{F + R + P_{\text{ctx}} + R_{\text{ctx}}}{4}$$

This composite score offers a balanced measurement of how well our system retrieves supporting evidence and generates a relevant, factual answer in the domain of policy / FAQ documents. We will also analyze component scores individually to identify weaknesses (e.g. low context recall might indicate retrieval gaps, while low faithfulness might indicate hallucination in generation).

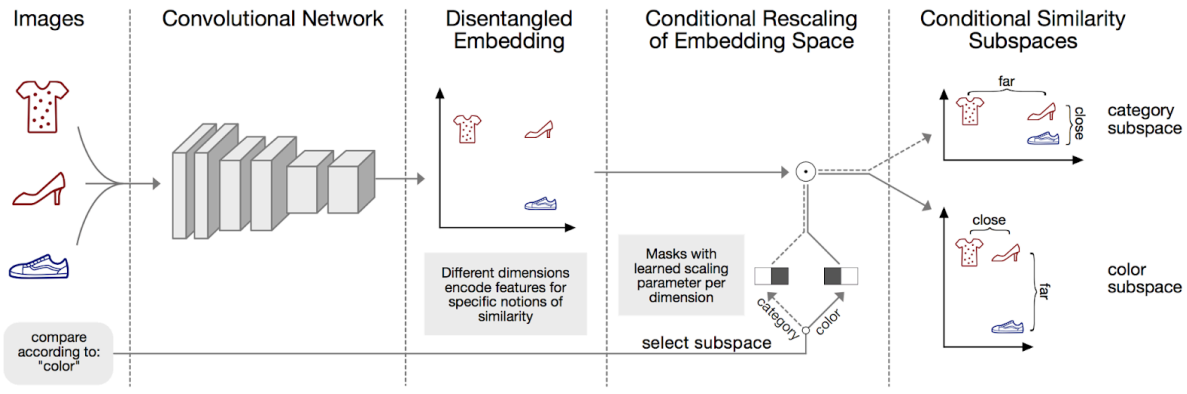
We will be aiming for a **RAGAS score of 0.8** or above which most RAG based applications are able to achieve. It has also been mentioned in the official RAGAS documentation:
https://docs.ragas.io/en/stable/concepts/metrics/overview/#metric-design-principles

Literature review for Product Rag:

1. NeuroStylist (https://liqiangnie.github.io/paper/p753-song.pdf)
    a. Used image as well as bag of style text input
    b. Trained to cluster matching top and bottom closer.
    c. Used triplet (input + pair + non-pair) items for calculating loss
    d. Used AUC and MRR metrics for performance

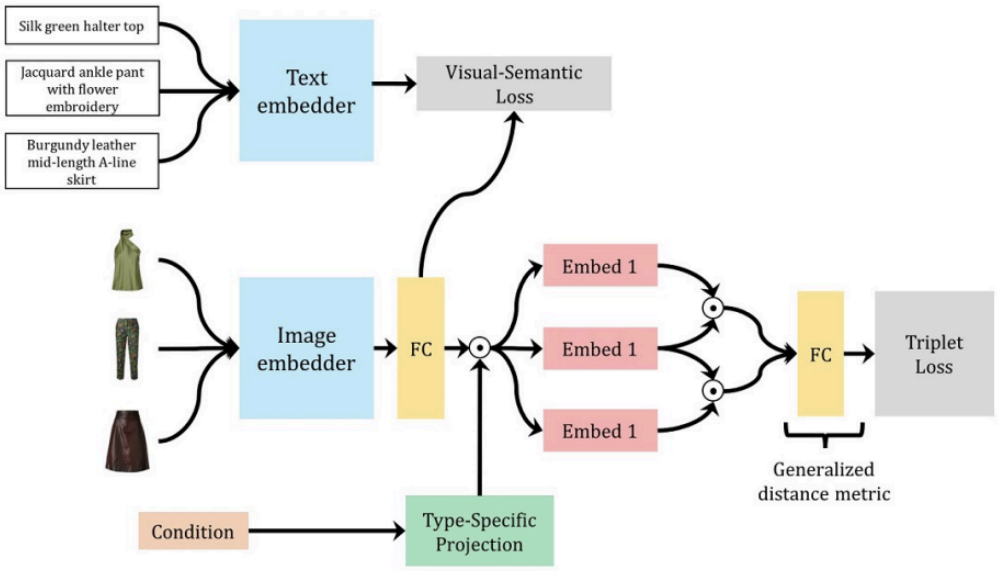


2. Conditional Similarity Networks (https://github.com/andreasveit/conditional-similarity-networks?tab=readme-ov-file
    a. Normally embedding the products clusters similar categories together.
    b. This research paper focuses on clustering compatible products along a specific dimension



3. Learning Type-Aware Embeddings for Fashion Compatibility (https://arxiv.org/pdf/1803.09196)

a. Used both image and text for training the model
b. Again triplets are used to make the model learn about compatibility



Inference from the literature reviews:

1. All the papers we researched focused on suggesting a good pair given a top/bottom and some models also took into account user preference. But no model tried to take into account user defined filters during search.
2. With respect to the dataset part, for filtering we require lots of features on which filters can be applied. The datasets which have features with it do not provide any compatible pairs data. If compatible pairs data is present, then features are not available.

So, our suggestion system is going to use text features alone and will suggest clothes based on user inputs.

Existing implementations based on which we identified gaps and scopes of improvement (Chatbot type implementations)

- **H&M "Virtual Stylist"**, experimental assistants / chatbots for style recommendations, virtual try-on etc.
- **Klarna's "Shop-with-me / AI Style advisor"**, Style recommendations, curated product browsing, conversational interface in app / web.

- **Yesplz AI**. Quiz based model where a set of questions is framed and products are shown based on the answers. A similar kind of approach can be used for filtering the suggestions.
- **PairFashionExplanation** — Explains how prompt engineering can be used to get matching pairs of clothes. The dataset used is fashion VC which has image + description.

Data Sourcing, Licensing, and Embeddings:

For the **catalog**, we are considering a dataset containing all product details along with images, so that when a customer requests a particular item, the AI can search the dataset and display the corresponding product image. The initial thought process is we will embed the catalog and store it in a vector database for retrieval.

For **policies**, we will refer to the guidelines of major online clothing brands like H&M, Zara,levis etc.as baseline references, and then customize them to generate a comprehensive internal policy document for the chatbot.

Both the FAQ's and policy documents will be converted into embeddings using models like Sentence-BERT (for text), and stored in a vector database for fast and accurate retrieval during chatbot queries.

Gaps and Opportunities in existing solutions:

- Existing solutions relied on multiple external websites instead of a single catalogue, causing agents to suggest clothes outside of our catalogue. Whereas our AI will use only the company's internal product catalogue with all item details and images. Recommendations will be grounded in this dataset, ensuring items are available and from our catalogue. This prevents promoting third-party products.
- **The agents did not stick to the customer request** (when asked for a pant, the agents suggested other products too). So we will implement query-based filtering in the retrieval

pipeline, using product metadata and embeddings to ensure the AI only selects items that match the customer's specific request.

- **The agents did not have much fashion sense**. Not always relevant products were shown. For example, when asked to suggest a pant for an already available red shirt, the agent did not pair a proper pant. Actually the pants suggested didn't even take into account the red shirt. So we will create sets of rules for a GPT, so that outfit suggestions are context-aware and items like pants are paired appropriately with the user's existing clothing.
- **The chat history is not maintained properly**. For example, we requested an outfit for a business meeting, and the agent suggested some blazers. Next we gave a prompt 'need a blue one'. Here the bot forgot about the occasion 'business meeting' and suggested even T-shirts.We will implement a vector-based conversational memory to retain context, ensuring follow-up queries consider previous prompts like occasion or item type.

2. Dataset Analysis

About the dataset (Policy Rag):

A comprehensive policy framework was developed using ChatGPT, drawing references from established policy documents of leading clothing retailers such as H&M, Marks & Spencer (M&S), and Levi's. The formulated document encompasses key operational and customer-centric policies, including exchange, refund, and return procedures, garment recycling initiatives, loyalty and reward programs, as well as gifting policies

About the Data (Product Rag):

1. Dataset Source

The dataset originates from a Kaggle competition organized by **H&M Hennes & Mauritz GBC AB** titled **"H&M Personalized Fashion Recommendations"**. The competition challenges participants to develop personalized product recommendation models using a variety of data sources, including:

- **Product metadata** (We are using this) - articles.csv
- **Product images** (We are using this)
- **Historical transaction data - transaction.csv**
- **Customer metadata** - customer.csv

2. License and Usage Terms

- The dataset is provided **exclusively for non-commercial purposes and academic research**.
- **Commercial use of the data is prohibited**.
- Sharing the data privately outside authorized teams is **not permitted**.

3. References

- H&M Kaggle competition page: https://www.kaggle.com/c/h-and-m-personalized-fashion-recommendations

We have following 3 tables now as csvs:

Articles.csv : contains the product catalog

Customers.csv: this contains 5 customers which have persona and few dummy customers which can be considered as POS machines or website checkouts.

Transactions.csv: This contains the purchases made by the customer.

Currently we have uploaded only sample images from the product Catalog, as we cannot upload all the images to github.

EDA On Policy's (Policies Rag)

Total number of words - **5146 words**

Number of policies **- 13**

Average word count per policy **- 396**

Word count per policy:

- Clothing Retail – Returns, Refunds & Exchange Policy - 43
- General Return & Exchange Overview - 760
- Refund Eligibility - 622
- Eligibility for Exchange - 683
- Order Tracking - 433
- Delivery Policy - 467
- Order Cancellation Policy - 534
- Accepted Payment Methods - 433
- Taxes (GST) and Pricing Transparency - 80
- Account Signup Policy - 272
- Registered Address and Legal Policy - 207
- Garment Recycling Program - 366
- StyleRewards Loyalty Program Policy - 246

Summary Stats:

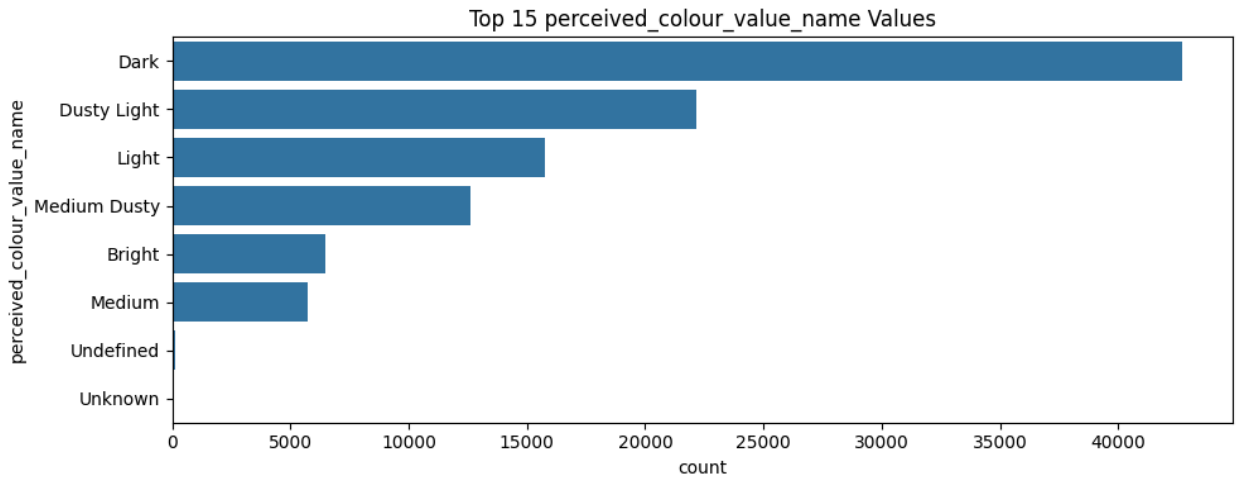
Minimum policy word count - 43

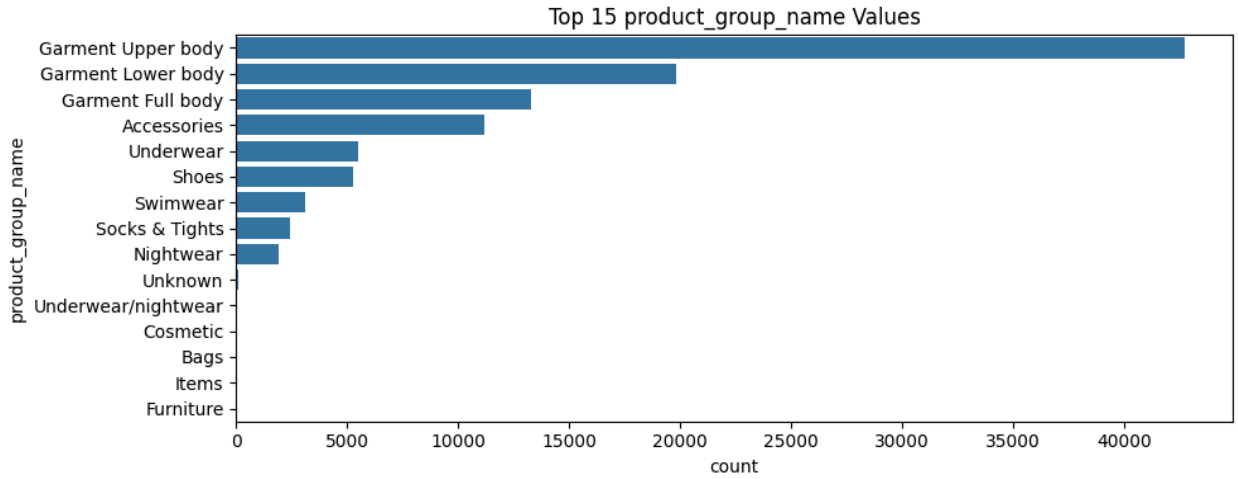
Max policy word count - 760

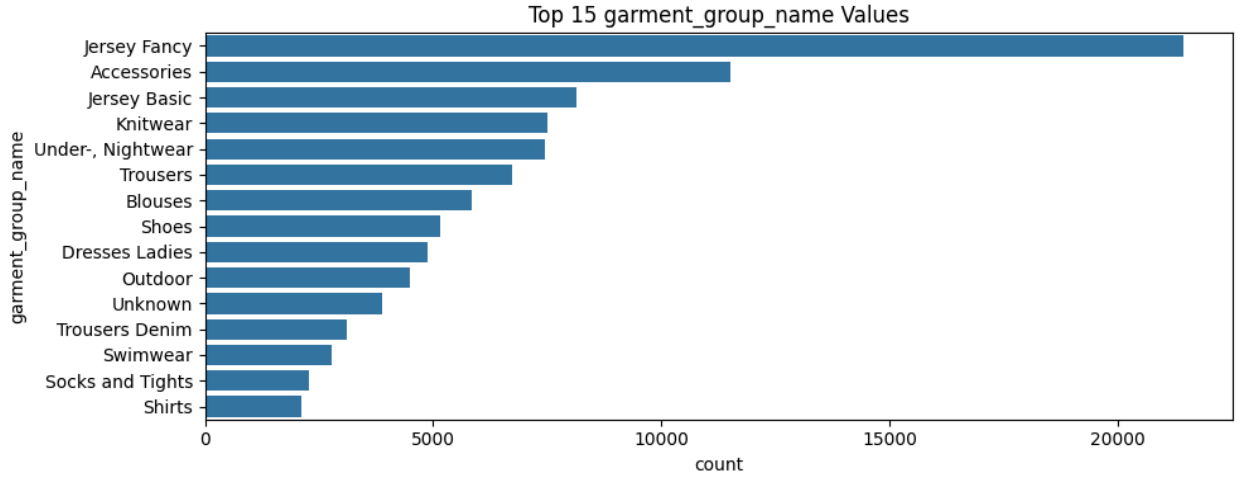
Median policy word count - 433

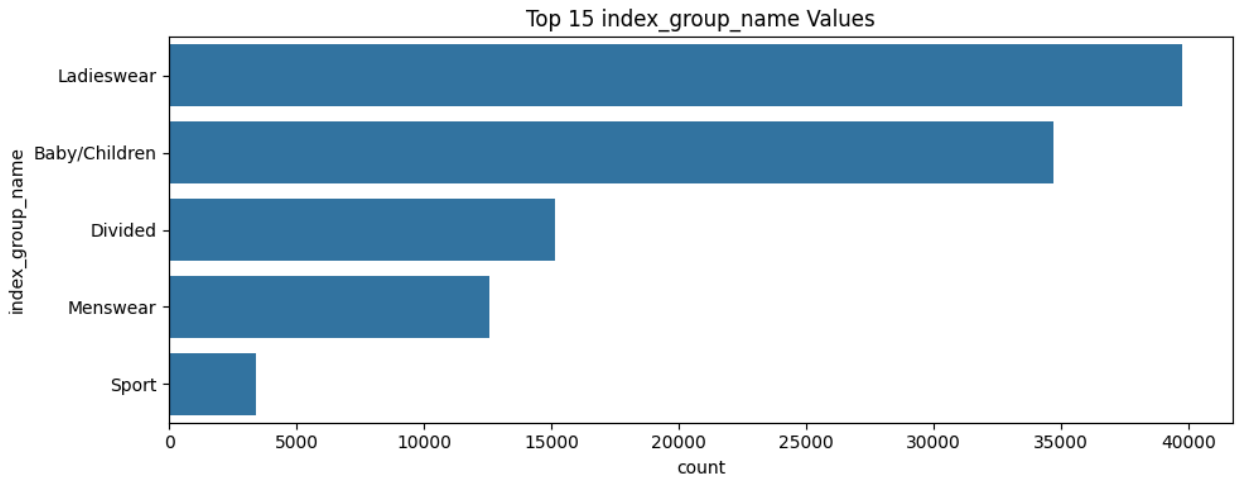
Mean/average policy word count - 396

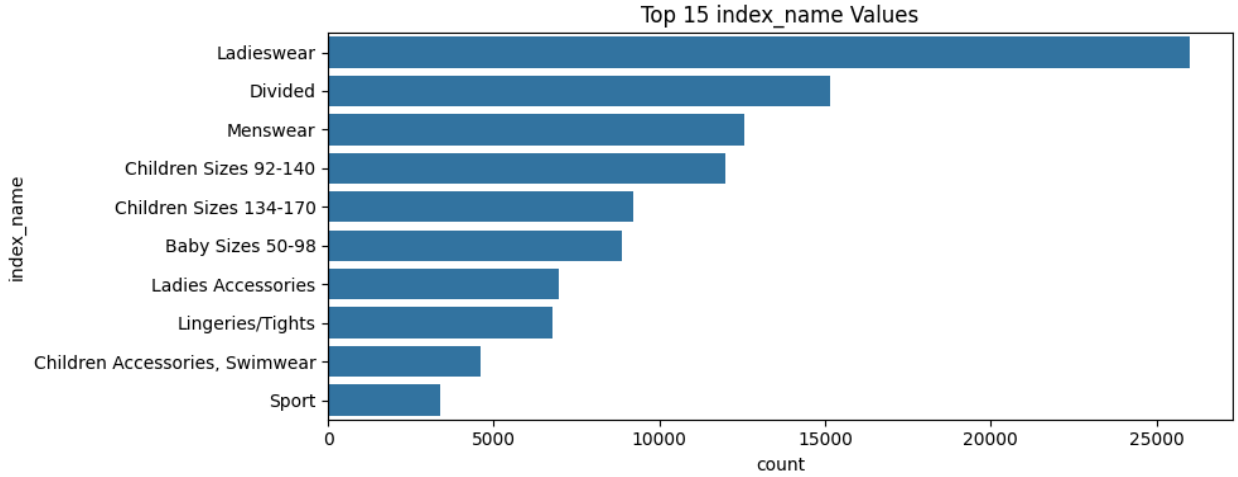
Most common range for each policy- 350-500

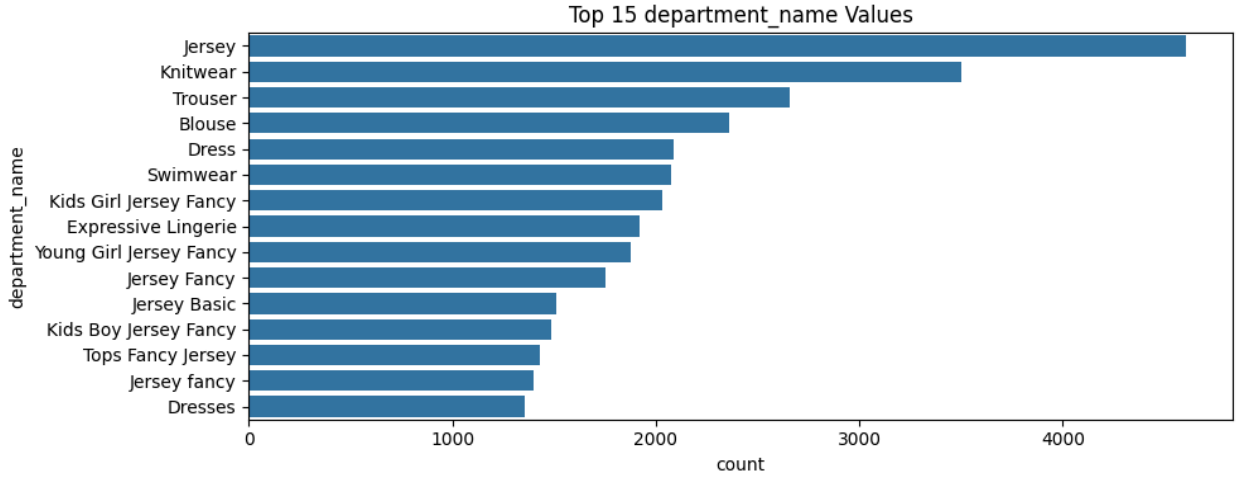

Top 15 perceived_colour_value_name Values
perceived_colour_value_name
Dark
Dusty Light
Light
Medium Dusty
Bright
Medium
Undefined
Unknown
0
5000
10000
15000
20000
25000
30000
35000
40000
count

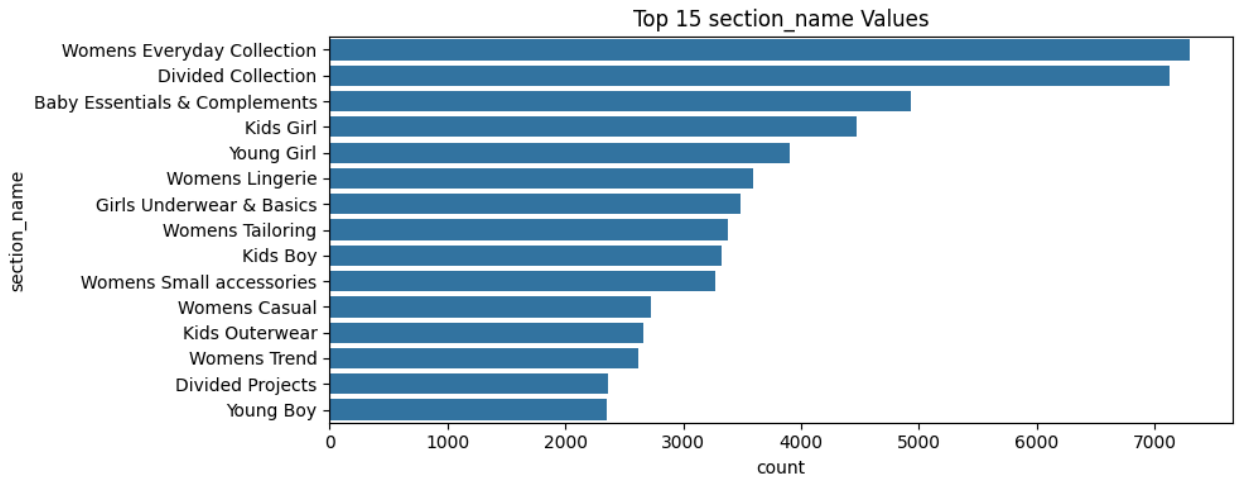

Top 15 product_group_name Values
product_group_name
Garment Upper body
Garment Lower body
Garment Full body
Accessories
Underwear
Shoes
Swimwear
Socks & Tights
Nightwear
Unknown
Underwear/nightwear
Cosmetic
Bags
Items
Furniture
0
5000
10000
15000
20000
25000
30000
35000
40000
count


Top 15 garment_group_name Values
garment_group_name
Jersey Fancy
Accessories
Jersey Basic
Knitwear
Under-, Nightwear
Trousers
Blouses
Shoes
Dresses Ladies
Outdoor
Unknown
Trousers Denim
Swimwear
Socks and Tights
Shirts
0
5000
10000
15000
20000
count


Top 15 index_group_name Values
Ladieswear
Baby/Children
Divided
Menswear
Sport
index_group_name
0
5000
10000
15000
20000
25000
30000
35000
40000
count


Top 15 index_name Values
Ladieswear
Divided
Menswear
Children Sizes 92-140
Children Sizes 134-170
Baby Sizes 50-98
Ladies Accessories
Lingeries/Tights
Children Accessories, Swimwear
Sport
index_name
0
5000
10000
15000
20000
25000
count


Top 15 department_name Values
Jersey
Knitwear
Trouser
Blouse
Dress
Swimwear
Kids Girl Jersey Fancy
Expressive Lingerie
Young Girl Jersey Fancy
Jersey Fancy
Jersey Basic
Kids Boy Jersey Fancy
Tops Fancy Jersey
Jersey fancy
Dresses
department_name
0
1000
2000
3000
4000
count



What is RAG?

RAG, or Retrieval-Augmented Generation, is a cutting-edge artificial intelligence technique that enhances the capabilities of large language models (LLMs) by connecting them to external knowledge sources. In essence, it allows these models to "look up" information before answering a question, leading to more accurate, up-to-date, and contextually relevant responses.

Embedding Models:

- **Embedding Strategy and Model Selection: :** The RAG methodology depends on converting complex data (text and metadata) into numerical embeddings, which are stored in vector databases (VDBs) for rapid retrieval via semantic search.
- **Policy RAG Embedding:** The team selected the BAAI/bge-base-en-v1.5 (BGE) model as the preferred choice for Part A (Policy RAG). BGE is specifically optimized for dense retrieval in RAG systems, balancing high accuracy with efficiency for semantic search.
- **Preprocessing:** The policy documents underwent a specific chunking strategy: the text was first segregated based on sub-headings before being split into chunks. Each chunk was then embedded using the BGE sentence transformer and persisted in ChromaDB. Keywords (generated using Llama-3.1-8b-instant) and policy titles were also added as metadata to the chunks to enhance retrieval quality.
- **AI Stylist (Product RAG) Embedding:** While the immediate implementation utilized BGE models for text embeddings, FashionSigLIP was identified as the preferred future

model for the AI Stylist component. FashionSigLIP is a domain-tuned, multimodal architecture optimized for fashion and e-commerce imagery, offering the best balance of fine-grained fashion understanding and text–image alignment. This is critical for future extensions like searching based on user-uploaded images.

- **Preprocessing:** For the current Product RAG, unwanted columns (like index codes and category codes) were removed from the catalog data, and the remaining product details were combined, embedded using the BGE model, and saved in a Parquet file for use with a VDB (FAISS).
- **E5 Family (Embeddings from Instructions):**

E5 (from Microsoft & Intfloat) is based on **instruction-tuned contrastive learning**.Instead of only learning from sentence pairs, it learns from **instruction–response pairs**

- **Main goal -** To generate embeddings optimized for **retrieval and ranking** tasks. Unlike plain similarity models (like SBERT), E5 is **instruction-tuned**, meaning it understands "what is being asked" (query intent) and "what content can answer it" (passage meaning). This makes it ideal for **RAG pipelines** where user queries need to match relevant document chunks.
- **Trained with -** massive datasets from search engines, QA pairs, and web text, with explicit "query/passsage" formatting.
- **Strengths:** Open-source and can be used locally with good performance on commodity GPUs Handles diverse natural language phrasing, so it performs well even on reworded or indirect questions.

Designed specifically for **retrieval-based tasks** like RAG — excels at mapping questions to the right document sections.

Instruction-tuning makes it **query-aware**, improving precision in answer retrieval.

Limitations -

- Slightly heavier (~768 dimensions).
- Requires proper **prefix formatting** ("query:" / "passage:") for optimal results.
- Only 512 tokens

BGE Family (BAAI General Embedding):

Similar to E5 but tuned for **dense retrieval** efficiency.

- Main goal - To produce embeddings that are highly effective for **dense retrieval and semantic search**, while being efficient enough for large-scale RAG systems. It's optimized for **retrieval robustness** and maintains **semantic consistency** even when model size is small.
- Trained with Large multi-domain dataset including natural text, Q/A pairs, web docs, and multilingual sources.
- **Strengths:**
  - Specifically tuned for **dense retrieval in RAG**, where matching query and context meaning matters more than syntax.
  - Delivers **strong performance in multilingual and domain-agnostic document search**, making it versatile for global or diverse datasets.
  - Smaller models (like bge-small-en or bge-base-en) give **excellent performance-to-size ratio**, ideal for local or edge RAG setups.
  - Strong alignment with LLM-based question answering pipelines — integrates well with vector databases (like FAISS, Milvus, Chroma).
- **Limitations -**
  - Has slightly **less fine-grained contextual nuance** for long, descriptive paragraphs compared to larger OpenAI models.
  - Performance may drop if text is very abstract or literary (e.g., philosophical or emotional context).

- **OpenAI Embedding Models (text-embedding-3-small / large):**
  - **Main goal -** To provide **universal, domain-general embeddings** for text, code, and documents — designed to capture meaning, context, and relationships across

wide-ranging corpora. Perfect for **production-grade RAG** where precision and scale are critical.

- **Trained with** Massive multi-domain corpus: web text, documentation, code, Wikipedia, and instruction data. Models learn not only meaning but also *relations* and *metadata semantics*.
- **Strengths**
  - State-of-the-art semantic accuracy.
  - Handles long context windows.
  - Excellent for production-scale similarity and clustering.
- **Limitations -**
  - Cloud only (requires API).
  - Cost per token (though small models are very cheap).
  - No local interpretability or open weights.

- **FashionSigLIP:** FashionSigLIP is a domain-tuned version of Google's Sigmoid Loss CLIP architecture, optimized for fashion and e-commerce imagery.
  It retains CLIP's multimodal nature but is specifically trained to handle fine-grained style, texture, and material distinctions common in apparel data.
  - **Main goal -** To generate high-fidelity multimodal embeddings that capture **color, cut, fabric, and aesthetic "vibe"** of fashion items for recommendation, similarity search, and retrieval.(both image and text - multimodal)
  - **Trained with** Specialized **fashion e-commerce datasets** containing product listings, outfit metadata, and descriptive text from online catalogs.
  - **Strengths**
    - **Fine-grained style sensitivity** — can tell apart fabrics, necklines, and patterns within similar categories.
      • Excellent **cross-modal alignment** for both image→text and text→image retrieval.
      • **Lightweight and fast**, making it ideal for embedding large fashion inventories.

- **Limitations**
  - Slightly narrower domain — not suited for generic images (like landscapes or food).
    • Requires high-quality fashion metadata for best results.

- **DINOv2 (Meta AI):** DINOv2 (Self-Distillation with No Labels) from Meta AI is a **pure vision transformer**, trained via self-supervised learning.
  Unlike CLIP-style models, DINOv2 doesn't use text — it learns powerful visual representations directly from unlabeled image data.
  - **Main goal -** To produce robust visual embeddings that capture **style, structure, and visual patterns** without needing captions or annotations.
  - **Trained with** - Billions of images from diverse visual domains, using a **self-distillation** process that forces the network to learn meaningful image clusters and invariances.
  - **Strengths**
    - **Exceptional visual embedding quality** for clustering, similarity, and unsupervised grouping.
      • Handles fine-grained visual differences (textures, shapes, symmetry).
      • **Very efficient** and flexible for downstream visual tasks.
  - **Limitations –**
    - No text encoder — cannot perform text→image retrieval ("red dress" won't work directly).
      • Requires separate text model or cross-modal alignment for multimodal tasks.

- **FashionCLIP (by Marqo):** FashionCLIP is a domain-specialized adaptation of CLIP trained exclusively on **fashion datasets**, focusing on apparel, accessories, and footwear.
  It aligns fashion product descriptions with corresponding catalog images to learn embeddings that understand **style, silhouette, and trend**.

- **Main goal -** To provide fashion-aware multimodal embeddings that accurately represent both the **visual look** and **descriptive language** of clothing and accessories.
    - **Trained with -** Large-scale **fashion industry data** (images + descriptions) curated from product catalogs and style-focused datasets.
    - **Strengths –**
        - Excellent understanding of **fashion attributes** like silhouette, material, and seasonality.
            • Text and image embeddings are **well-aligned** for descriptive queries.
            • Designed for fashion-specific retrieval, style search, and recommendation systems.
    - **Limitations –**
        - Smaller training set compared to SigLIP — slightly weaker **cross-domain generalization**.
            • **Slower inference** for very large catalogs.
- **Preferred embedding model:**
    - Part A – **BGE family -** because they are specifically optimized for dense retrieval in RAG, offering a strong balance between accuracy, efficiency, and local usability. They provide high-quality semantic matching while remaining lightweight enough for scalable, on-device retrieval.
    - Part B – **FashionSigLIP -** because it offers the best balance of fine-grained fashion understanding and text–image alignment, unlike FashionCLIP which has limited generalization and DINOv2 which lacks text comprehension.
- **Re-embedding:**
    - When changes are made in the embedding doc we will simply delete all the embeddings generated for the doc from the vector database (delete the collection) and re-embed the doc again.
    - With respect to the products, if a certain product is removed from the catalog, we will just delete the corresponding embedding.
    - If any product's info is edited, we delete the embedding of that product and re-embed it.

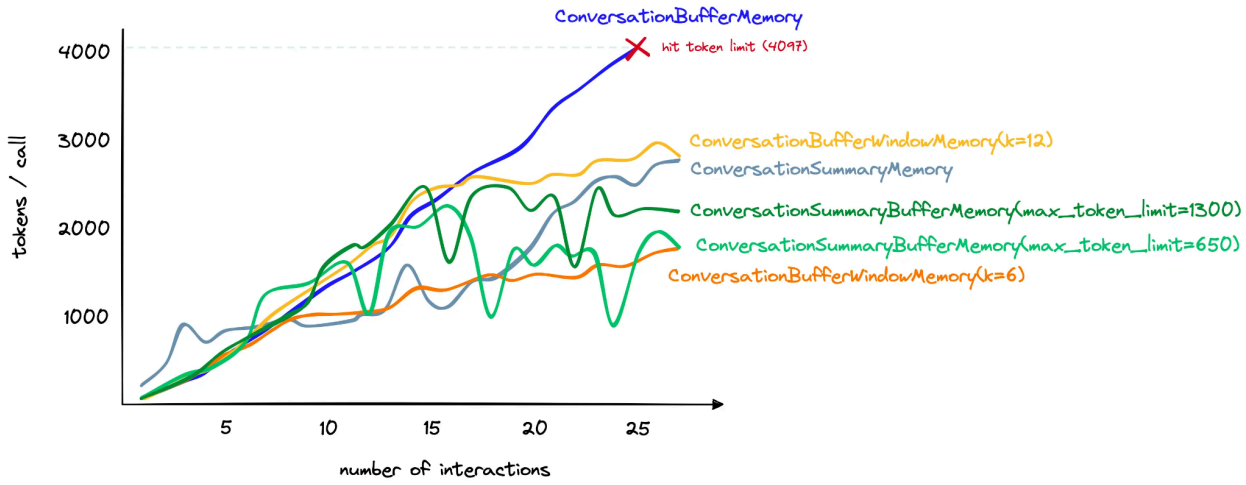
- **How to maintain chat history and feed to LLM?**

  Conversational history : For implementing this task in our chatbot, a conversational memory will be maintained, enabling a coherent conversation.

  Breakdown of the pipeline-

  - Initialize the *ConversationChain* with a LLM model.
  - We will be keeping the conversational memory in one of the parameters for the model in *{history}*, and
  - For saving each customer's chat history from a session to a persistence storage, we will be using Redis (planning to save latest session's chat history)
  - Since maintaining long chat histories of customers could lead to exceeding the maximum token limit, we will be implementing conversation summarizing techniques.

    The difference between such techniques *(e.g. ConversationSummaryMemory)* and without it *(ConversationBufferMemory)* is shown in the chart below.



- **Context-Aware Query Flow**

  The core methodology implements a multi-stage flow centered on context building and LLM classification:

  - **Query Classification:** An initial LLM determines if the user's input is related to Policy, Return, or Product Search.
  - **Context Building:** For product searches (AI Stylist), extensive customer context is assembled, including age, gender, location, and previous purchase details (style, price point, etc.).
  - **Mandatory Information Check:** The pipeline checks if mandatory information (like age and gender) is available for the search. If this context is missing, the LLM will explicitly ask the customer a follow-up question before proceeding.
  - **Memory Management (Planned):** To handle lengthy conversations and mitigate latency caused by exceeding token limits, the architecture includes plans to use conversation summarizing techniques, such as ConversationSummaryMemory, possibly utilizing Redis for persistent session storage.

3. Architecture

- **Login and Log out:**
  - To login, the user must give a prompt in the ChatBot in this format "cust_id:{customer_id}". e.g. – "cust_id:1", this will lead to logging in as customer 1.
  - To log out, the user needs to type "logout" in the ChatBot.
- **Context:** Must be built before answering any user query. If context is missing, the system MUST ask for it — it cannot proceed.The context contains all the user details which are available in our database. This is maintained to get all the details

about the customer in 1 place, so that there is no need to fetch the customer frequently. What is included in context:

1. User is Logged in

Context consists of three main parts :

- Customer Details (Columns: age, gender, size, cust_id). The system fetches data automatically from CSVs using pandas.
- Transaction History (Columns: t_dat, SKU_No, payment_method, prod_return_type / return_Type, return_Status, delivered)
- Product Details (Columns: SKU_No, prod_name, product_type_name, product_group_name,
  colour_group_name, department_name, Price_INR, Discount_Percentage, Return_Type, Eco_Score)
- Valid customer IDs – 1,3,5.

2. User is logged out

- The LLM must explicitly ask for age and gender before proceeding.
  Once that information is provided, a minimal context is created using those values.

- **AI Stylist:** The **AI Stylist** is responsible for understanding the user's fashion intent and recommending the most relevant clothing products. It personalizes results using the **customer's context** (age, gender, purchase history, etc.) and **semantic understanding** of the user query. Its done in the following 3 3 steps:
  - **1. Query Refinement (refine_prompt) :** The user query and customer context are passed to a dedicated LLM prompt called **refine_prompt**. Its purpose is to **rewrite the user query** into a refined, detailed version that takes into account user-specific attributes.

Example:

```
Gender: Female
Age: 25-30
Postal Code: Mumbai
--- PURCHASE TRANSACTIONS ---
Transaction 0: Purchased formal shirts and trousers
--- PRODUCTS PURCHASED ---
Product 0: Cotton Slim Shirt, Color: Blue
Product 1: Linen Regular Fit Pant, Color: Beige

User Query:
show me something to wear to a party

Refined Query:
show me trendy but elegant party outfits for a 25-year-old female from Mumbai,
preferably in styles similar to her past purchases like shirts and pants, in blue or beige tones.
```

2.

Searching Product Catalog (search_products): The **refined query** and all product descriptions are converted into **numeric embeddings** using the **BGE model**.

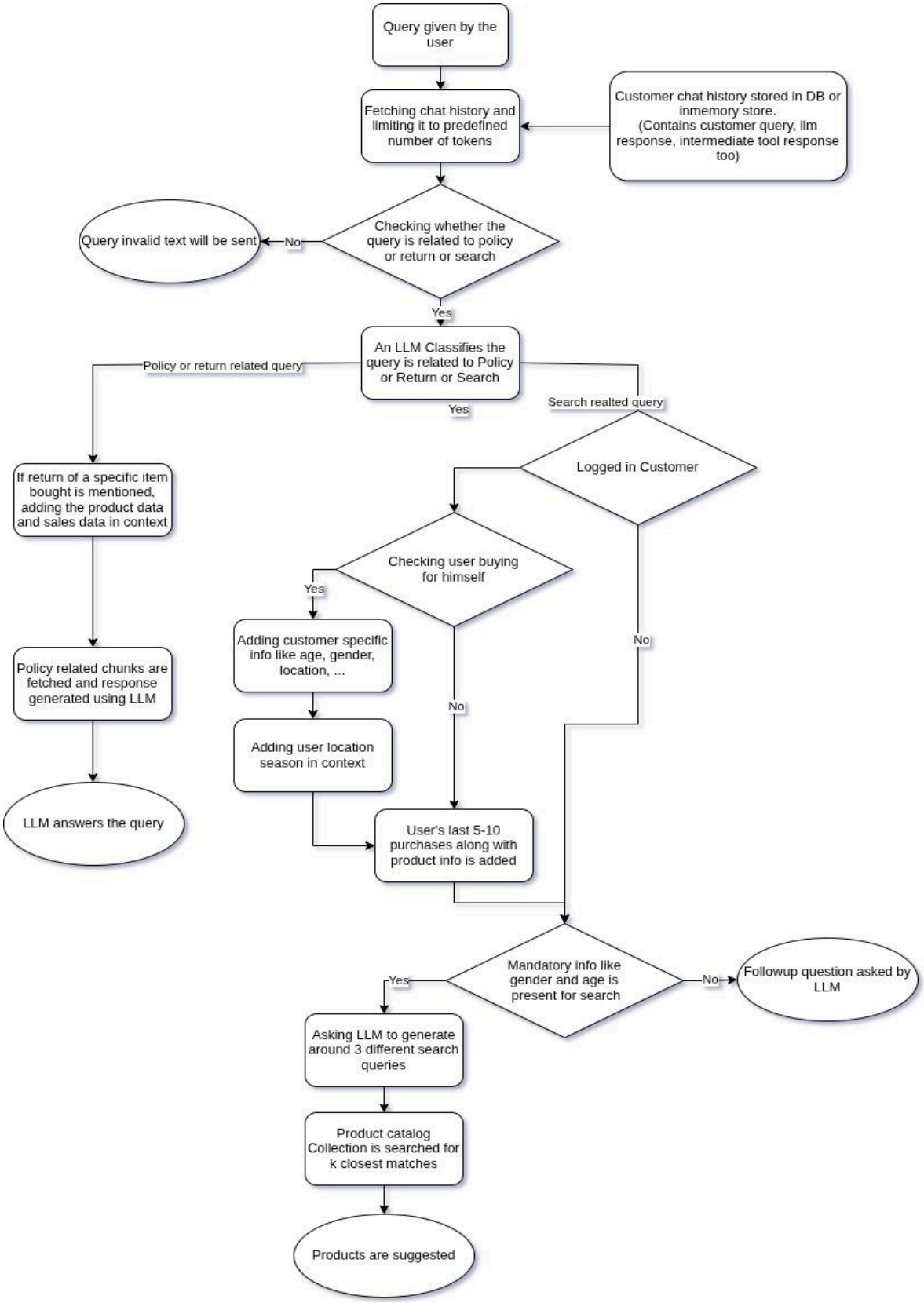
Then the system:

- Computes **cosine similarity** between the query embedding and every product embedding.
- Measures how semantically close each product is to the query intent.
- Sorts all products by similarity score.
- Returns the **top 20 most relevant products**.

This ensures that results are driven by **semantic meaning**, not just keywords.

- **3. Ranking the Top 3 (recommend_top3_structured):** The top 20 products and the refined query are passed to another LLM called recommend_top3_structured, whose goal is to identify the top 3 most suitable products for the user.
- **Weightage Logic:**
- 80% → Current user query meaning (highest priority).
- 15% → Customer context (e.g., gender, location, preferences).
- 5% → Purchase history (style consistency only).

This prevents bias from past purchases — e.g., a user who bought party wear earlier but now asks for office wear will get relevant new options. The LLM outputs the final top 3 products in a structured format.


Query given by the user
Fetching chat history and limiting it to predefined number of tokens
Customer chat history stored in DB or inmemory store. (Contains customer query, llm response, intermediate tool response too)
Checking whether the query is related to policy or return or search
No
Query invalid text will be sent
Yes
An LLM Classifies the query is related to Policy or Return or Search
Policy or return related query
Yes
Search realted query
If return of a specific item bought is mentioned, adding the product data and sales data in context
Logged in Customer
Checking user buying for himself
Yes
No
Adding customer specific info like age, gender, location, ...
Policy related chunks are fetched and response generated using LLM
Adding user location season in context
No
LLM answers the query
User's last 5-10 purchases along with product info is added
Mandatory info like gender and age is present for search
Yes
No
Followup question asked by LLM
Asking LLM to generate around 3 different search queries
Product catalog Collection is searched for k closest matches
Products are suggested

Flow Explanation:

Checking whether the query is related to policy or return or search:

LLM will be fed the user query along with history. It will also be fed an instruction that it is a chatbot which will answer about policies of an online clothing store or it will help customers search for clothes. If the query along with the history is related to the tasks provided to the LLM, we will move further else the LLM will ask the customer to ask relevant questions.

An LLM Classifies the query is related to Policy or Return or Search:

- Here LLM is instructed to classify the query into one of the three tasks. The query asks for any info about store policy or any return instruction or it is a search query for clothes. Based on the classification provided by the LLM we will choose a path and move further.
- If the query is a simple policy related query, we will add context via embeddings retriever.
- If the query is related to returning a product, then the product is searched in the product catalog and also the sales data is added to the context along with the context we will be getting from searching the policy doc via embeddings retriever.
- For product search related queries lots of other context needs to be added which will be explained in the following sections.

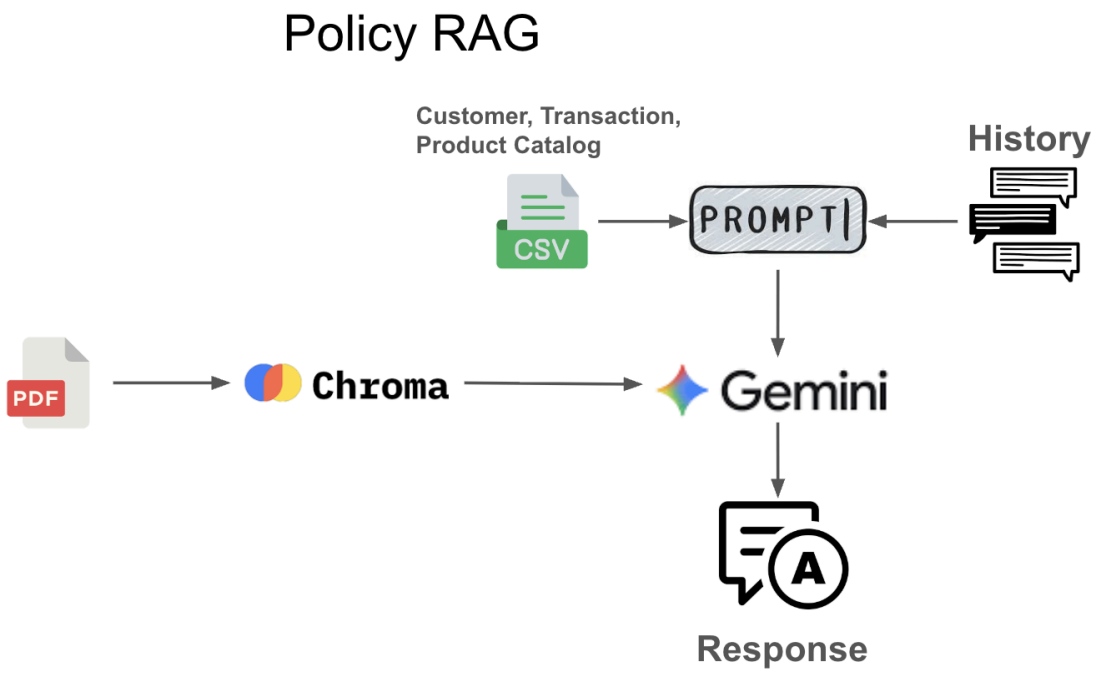
Context building for product search:

- If the customer is a logged in user, then we can add his/her data like age and gender in context if he is buying for themself.
- If the customer is a logged in user and not buying for themself, we will try to get age and gender from the query, if not possible, the LLM will ask a follow-up question. The same approach will be used in case of guest users.
- If we are able to get location data (can get it from user DB), an api call to fetch the season of a particular zipcode can be used. This season info can be added into context.
- For logged in customers we also have previous sales data. Using that data we can add into context, at what price point the customer purchases products, how much discount was there on the products, category, sub category and style of products purchased, etc.
- If from the context built if all the mandatory info is present, we will ask LLM to come up with a search query using the context and search for relevant products in Vector DB. If

mandatory info is not available in context then a followup question will be asked by the LLM.

Search in Vector DB elaborated:

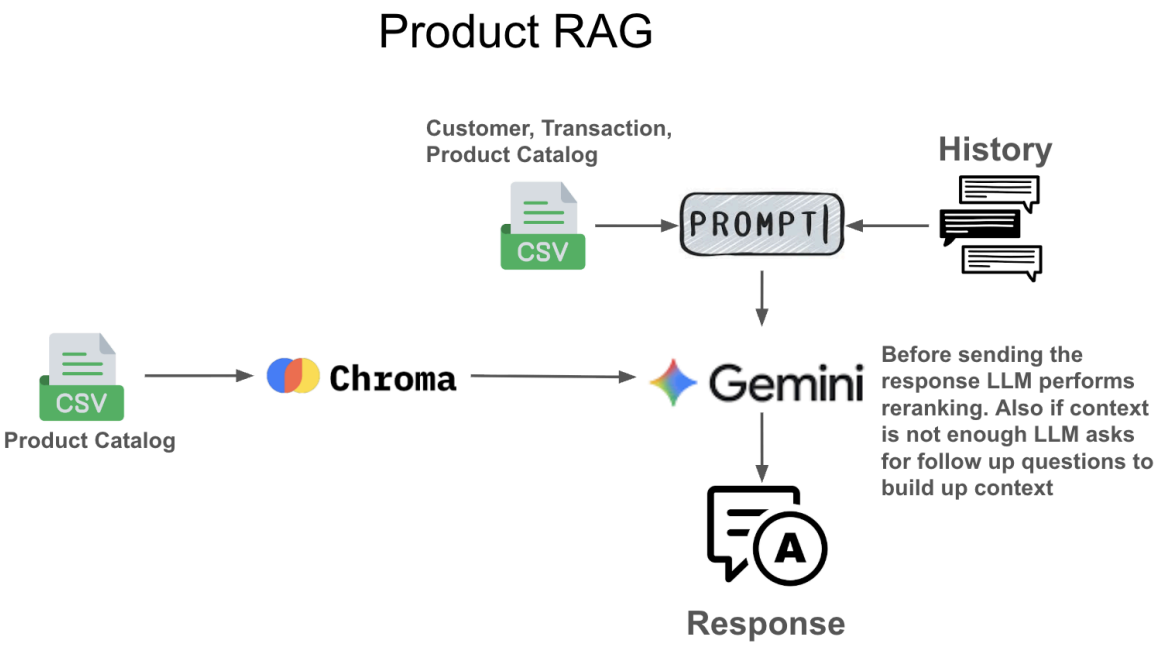
Policy Doc Search:



- We will search for lets say 5 chunks from policy doc to answer user query.
- The LLM will check whether the chunks are valid and remove invalid chunks.
- Also the LLM will check whether there is enough context to answer the user query. If not then more chunks will be fetched to answer the query (We can cap the total number of search by 3)

Product Catalog Search:

- The LLM will give out a search query and ask for matching products to the VDB.
- Then judging will be done whether the products given matches the search query or not (by an LLM).
- If not enough products (say 5) are relevant, then LLM will be made to make adjustments in search query and the VDB search will be triggered again based on the irrelevant products got (using the irrelevant product fetched, the LLM can identify what was the mistake in the search query and try to rectify it).
- This back and forth operation will be tried 2-3 times after which the products deemed irrelevant will be shown (Most probably because the product user is searching for is not available in the catalog).
- After we have the products which we want to show the customer, we will rerank that using the product info and context built. (this determines the order in which we need to show the search results)
- Currently we think we can use an LLM to do it. But we need further analysis in this part and will decide the approach in future.
- Currently we are thinking of a strategy, in which we embed the text columns of Product catalog and images seperately and concatenate them.



4. Model Development and Hyperparameter Tuning

- User context (age, gender, purchase history) is incorporated into query refinement for semantically relevant retrieval.
- Hyperparameter tuning explored embedding chunk sizes, overlap words, retrieval strategies, and LLM creativity parameters such as temperature and Top-K selections.
- Evaluation showed that the BAAI/bge-base-en-v1.5 model with specific chunk and overlap parameters optimized performance.
- The AI stylist follows a three-step process: refining user queries, product catalog search via cosine similarity of embeddings, and reranking top results with weighted customer context for personalized recommendations.

Hyperparameter Combinations

The following table summarizes the evaluated hyperparameter combinations for product recommendations:

Hyperparameter Combinations	Temperature	Top-K	Refine Prompt	Rerank Prompt
Combination 1	0.2	5	refine_prompt_contextual	rerank_prompt_contextual
Combination 2	0	10	refine_prompt_precise	rerank_prompt_precise
Combination 3	0.5	12	refine_prompt_balanced	rerank_prompt_balanced
Combination 4	0.8	20	refine_prompt_creative	rerank_prompt_creative

Query	Products	combination 1	combination 2	combination 3	combination 4
1	1	8	9	9	9
	2	4	8	7	6
	3	3	4	4	8
2	1	8	8	8	8
	2	7	9	8	9
	3	8	6	7	7
3	1	8	8	8	9
	2	8	9	9	8
	3	6	7	7	9

	Combination 1		
Query	**Product 1**	**Product 2**	**Product 3**
Q1 – “What about a red one?”	9 (NILS – Red Shirt → right color, ever	8 (SAILOR FRONT PRINT – Red Sweater	5 (CORN CLASSIC TEE – Dar
Q2 – “help me find a blue one”	10 (Retro Slim – Blue Denim → perfec	9 (Skinny 5pkt Trash 1 – Blue Denim → ac	8 (Skinny 5pkt Midnight – Dark
Q3 – “blue jean around 2000 INR”	10 (Skinny 5pkt Midnight – Slim fit and	9 (Skinny 5pkt Premiumprice – Blue → clo	8 (Retro Slim – Blue → fit a bit
Q4 – “red top”	10 (SAILOR FRONT PRINT – Red Sw	6 (Rosie Blouse – Black → cozy fabric but	7 (SAILOR FRONT PRINT – W
Q5 – “shirt that goes with cream pant”	10 (STYLO SLIM SHIRT – White → pr	7 (Sam LS BD – Light Blue → close tone b	4 (Pat LS BD – Black → color c
Q6 – “formal shirts for office meetings”	10 (Sam LS BD – Light Blue Shirt → e	6 (SPEED Paprika – Grey Sweater → casu	4 (SPEED Paprika – Black Sw
Q7 – “casual wear like polos or chinos”	10 (Rawley Chinos Slim – Yellowish Br	5 (Reggie Chino Shorts – Black → casual	4 (Slim Straight 5pkt Midway –
Q8 – “stylish yet comfortable for casual outings”	10 (Bernie – Light Blue Top → ideal ma	8 (Tilda Gatherings Top – White Sweater	4 (SPEED Paprika – Black Sw
Q9 – “light summer dress for office & evening”	10 (Topi Dress J – Black → perfect ma	6 (Polly Dress – Light Blue → wrong color,	5 (Sam Dress – Off White → n
Q10 – “trendy tops in pastel colors for brunch”	10 (Bernie – Light Blue Top Garment →	8 (Tulip Jumper – White → close lightness	6 (Tulip Jumper – Grey → neut

Averaged across queries and combinations.

Combination-3 (T=0.5 *T*=0.5, Top-K=12) achieved both high consistency and accuracy as rated by human evaluators.

Product RAG Scores

For product recommendations, scores were assessed across ten typical queries and all four hyperparameter combinations, using a ten-point scale. Sample results:

Combination 1:

```
Temp: 0.2
Top-K: 5
Refine Prompt: refine_prompt_contextual
Rerank Prompt: rerank_prompt_contextual
```

Combination 2:

```
Temp: 0
Top-K: 10
Refine Prompt: refine_prompt_precise
Rerank Prompt: rerank_prompt_precise
```

Combination 3:

```
Temp: 0.5
Top-K: 12
Refine Prompt: refine_prompt_balanced
Rerank Prompt: rerank_prompt_balanced
```

Combination 4:

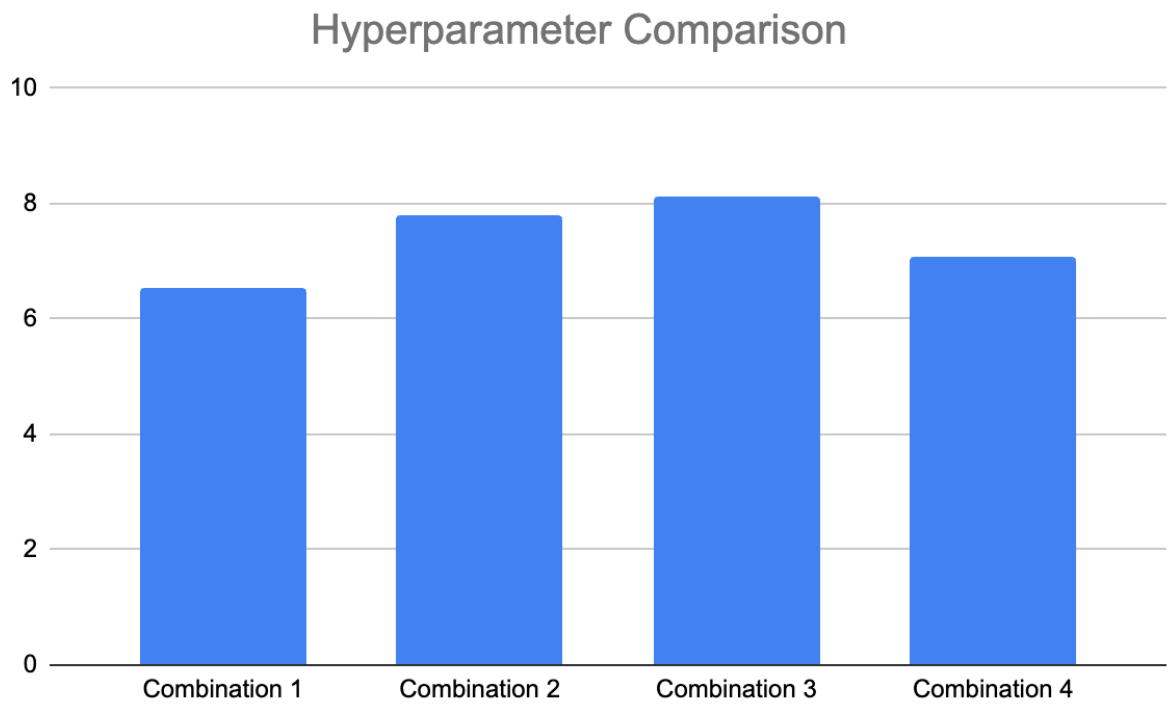
```
Temp: 0.8
Top-K: 20
Refine Prompt: refine_prompt_creative
Rerank Prompt: rerank_prompt_creative
```

Scoring:

- The chat bot results in three product suggestions. So, the setup is we rated the three product suggestions and the average of the three suggestions we will use as query score and use the same to compare with other hyper parameter combinations
- We performed scoring manually on the results produced with different sets of hyperparameter configurations. Also we tried generating scores using LLM to get a better picture. But for final comparisons we will use manual scores as it is more reliable.

Evaluation & Analysis

- Product RAG was evaluated on manually scored test queries focusing on relevance, filter adherence, history usage, and fashion sense.
- Four hyperparameter combinations were tested, where a balanced temperature (0.5) and Top-K (12) setting achieved the highest average score of 8.13/10, balancing creativity and precision.



From the graph we can see that combination 3 is performing better. So, when both creativity and precision are given importance, the RAG pipeline performed well.

Best Combination

```
Temp: 0.5
Top-K: 12
Refine Prompt: refine_prompt_balanced
Rerank Prompt: rerank_prompt_balanced
```

Please refer this sheet for more information

Sample test result:

```
Temp: 0.2
Top-K: 5
Refine Prompt: refine_prompt_contextual
Rerank Prompt: rerank_prompt_contextual
Query1:
```

Product 1:
• Name: SAILOR FRONT PRINT
• Type: Sweater
• Color: Red
• Price: 3821 INR
• Discount: 28%
• Return Type: 7 days return
• SKU: N/A

Product 2:
• Name: CORN CLASSIC TEE
• Type: T-shirt
• Color: Dark Grey
• Price: 3157 INR
• Discount: 24%
• Return Type: No return
• SKU: N/A

Product 3:
• Name: SPEED Paprika
• Type: Sweater
• Color: Grey
• Price: 2533 INR
• Discount: 53%
• Return Type: 15 days return
• SKU: N/A

The same way we run and document the results. Please refer to this document for full runtime logs. To see various

Policy RAG development and hyperparameter tuning:

Parameters chosen:

Model: Embedder which is used to embed the doc

Chunk size: The max word length of chunks that will be embedded

Overlap: The number of overlapping words between consecutive chunks

Retrieval strategy: One collection will be used / two

Combination 1:

```
Model: BAAI/bge-base-en-v1.5
Chunk size: 700 (collection 1), 300 (collection 2)
Overlap: 70 (collection 1), 30 (collection 2)
Retrieval: Both collections are queried and chunks with high cosine similarity is chosen.
Comparison done on results from both collection
```

Combination 2:

```
Model: all-MiniLM-L6-v2
Chunk size: 700
Overlap: 70
Retrieval: normal one collection setup
```

Scoring: We are doing manual scoring at this stage. We have the information about the availability of context for the test queries and check if context is fetched properly. Also we finally check whether the LLM answers properly using the provided context.

Observations:

Scoring is out of 5

Query	combination 1	combination 2
1	5	3
2	4.5	4.5
3	5	4
4	5	3
5	4.5	0
Average	4.8	2.9

- BAAI/bge-base-en-v1.5 performed well.
- Regarding finding out invalid queries, our RAG pipeline performed well without any need for hyperparameter tuning. Also, 100% of the time it found out when age or gender were missing.
- Please refer to this sheet for more information.

Sample test result:

--- Query 2 ---

Query: can i get discount voucher if i return old clothes

Context: ng way to recycle their used garments. By returning used clothes to our stores for responsible recycling or repurposing, customers contribute to lowering carbon emissions, conserving natural resources, and supporting a circular fashion economy. 2. Policy Overview Customers are invited to bring any brand of used or unwanted clothing, shoes, or textiles (in any condition) to participating stores. In return, they will receive a 15% discount voucher applicable toward the purchase of new clothing on their next visit. 3. How It Works 1. DropOff: Customers may drop their used textiles at designated Recycling Collection Boxes located near store entrances or customer service desks. Processed for the product amount only. The following are

nonrefundable: ● Shipping charges ● Platform convenience fees ● Giftwrapping or special service charges However, if a product is damaged or defective due to Clothing Retail's fault, shipping or convenience fees may be refunded upon request within 7 days of refund credit. 5. Refund Conditions Refunds will be approved only when: ● The returned product is in its original condition, with tags, packaging, and invoice intact. ● The item passed a quality inspection at our warehouse. ● No damage, alteration, or signs of usage are found postdelivery. ● Accessories or bundled items (if applicable) are returned together. Any returns will not be accepted. ● For electronics or watches, refunds are processed after the item passes quality inspection at the brand warehouse. Failure to comply with the above may result in rejection of the return request. 5. Return Process 1. Log in to your Clothing Retail account. 2. Go to "My Orders" and click on Return/Exchange for the respective item. 3. Choose Return or Exchange, specify the reason, and upload supporting images (if damaged). 4. Select your preferred refund mode (original payment method or Clothing Retail Wallet). 5. A pickup will be scheduled within 4–7 days or instructions for selfshipping will be provided. 6. Once the product passes the qual accessories or bundled items (if applicable) are returned together. Any product found in violation of these conditions will be deemed ineligible for refund. 6. Refund for NonReturnable Products Certain categories are nonreturnable (e.g., innerwear, accessories, jewelry, or electronics marked "NonReturnable"). Refunds for these will only be approved if the product was: ● Defective, ● Damaged in transit, or ● Incorrectly shipped. Such refunds are processed after internal verification and approval by our quality team. 7. Refund Notifications You will be kept informed through: ● Email notifications ● SMS updates ● Order status in your Clothing Retail Account If there's case pickup is unavailable at your location, you may selfship the product to our warehouse; the shipping cost will be reimbursed as Clothing Retail credits, subject to policy compliance. ● Returned products must be unused, unwashed, unaltered, and in their original condition with all tags,...

Response: Yes, if you return used or unwanted clothing, shoes, or textiles to participating stores, you will receive a 15% discount voucher applicable toward the purchase of new clothing on your next visit.

Please refer to this file for policy rag for full logs

The development initial reliance on fixed chunking strategies, followed by iterative testing of embedding models, which ultimately led to the adoption of a refined, context-based chunking strategy.

Transition to Context-Based Chunking Strategy

- Despite the superior performance of the BGE model, the testing revealed a fundamental flaw in the fixed chunking methodology:
- **Observation of Failure:** During hyperparameter optimization, the project team found that the initial chunking strategy (where policies were not segregated based on sub-headings) did not work well.
- **Upgrade to Context-Aware Strategy:** Based on these results, it was concluded that the Policy RAG needed an upgrade to become context-aware.
- **Implementation of Context-Based Chunking:** The new strategy mandated that the policy document must be segregated based on sub-headings before chunking. The policy document, which was originally created internally and synthesized from the guidelines of major retailers (H&M, M&S, Levis), was split based on policy sections.

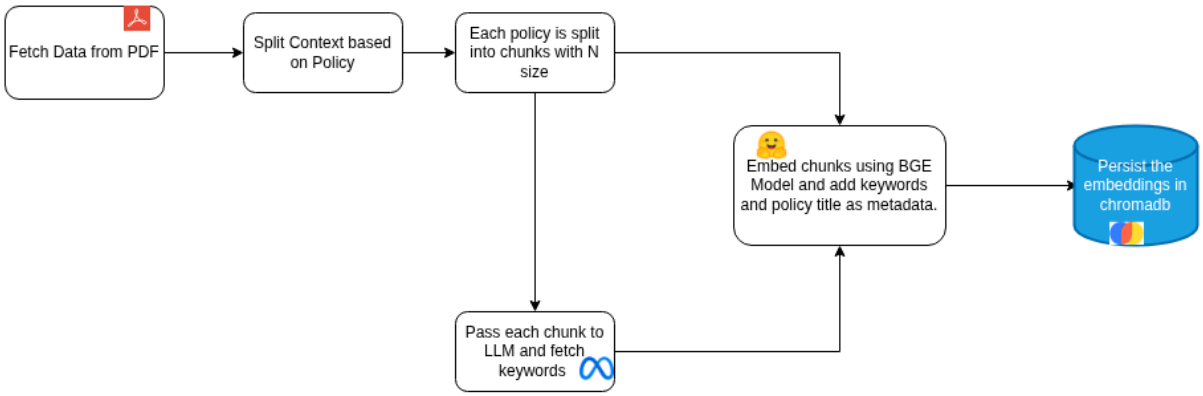
Final Processing Flow:

- The refined chunking strategy involves:
- Fetching data (e.g., from PDF).
- Splitting the context based on Policy.
- Splitting each segregated policy into chunks of size N.
- Passing each chunk to an LLM (llama-3.1-8b-instant was used in production) to fetch 5–7 keywords for metadata.
- Embedding the chunks using the BGE Model and adding the keywords and policy title as metadata.
- Persisting the embeddings in ChromaDB.

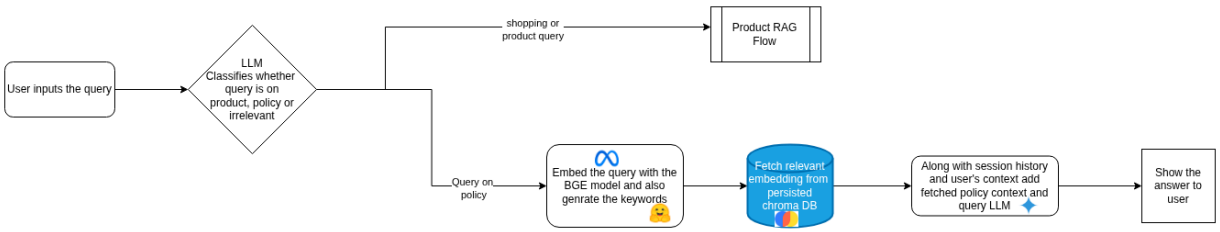
This transition significantly improved the system, ensuring that sufficient, relevant policy context was provided within each retrieved chunk, which facilitated more accurate response generation by the LLM. Chunk sizes above 100 words were found to be ideal in the new structure as they provided sufficient context

Based on the results it was concluded that there needs to be an upgrade in Policy rag and it needs to be context aware.

Following is the updated Chunking strategy.



Following is the updated Retrieval Strategy:



5. Evaluation Metrics for policy RAG (RAGAS)

System output quality was measured with the RAGAS framework, which captures key aspects of retrieval-augmented generation through the following metrics:

Ragas Metric	What it Measures	Significance of a High Score (Closer to 1
Faithfulness	**Fact-Checking (Hallucination)**	The generated answer is truthful and directly supported by the provided context documents.
Answer Relevancy	**User Intent**	The generated answer is fully relevant and directly addresses the user's question, without containing unnecessary or tangential information.
Context Precision	**Noise in Context**	The retrieved context documents are highly focused and contain minimal irrelevant or "noisy" information for answering the question.
Context Recall	**Completeness**	The generated answer incorporates all necessary facts from the context that are needed to provide a comprehensive response to the question.

Policy RAG Scores (RAGAS)

The policy assistant was evaluated with policy-related questions, scoring above 0.965 on faithfulness, answer relevancy, context recall, and context precision across diverse queries:

ragas_evaluation_data

6.Deployment & Documentation

1. Overview

For the deployment of our full-stack application, we initially attempted to use Render due to its simple workflow and free tier support. While the frontend deployment was successful, the backend failed because of high memory requirements.

To overcome this, we migrated our deployment to **Google Cloud Platform (GCP)** using a **Virtual Machine (VM)**.

Both the **frontend and backend** are now successfully deployed and running on the GCP VM.

2. Frontend Deployment (Render → GCP)

2.1 Initial Deployment on Render

- The frontend was deployed using Render's static site hosting.
- URL: **https://group-5-ds-ai-lab-project.onrender.com/**
- **Status:**
  - Deployment successful
  - Fully functional
  - Responsive and publicly accessible
- **Included components:**
  - Product search interface
  - Dashboard
  - Overall UI

Render handled the frontend smoothly because it required minimal computation and memory.

2.2 Current Deployment on GCP VM

- The frontend has now also been deployed on the **GCP VM** along with the backend.

- Running on the same VM instance ensures better integration and zero CORS issues.

3. Backend Deployment

3.1 Attempt on Render (Failed)

Backend URL: **https://group-5-ds-ai-lab-project-1.onrender.com/**

Status: **Failed to run**

Reasons for Failure:

- Our backend includes:
  - A large machine learning model
  - Embedding vectors
  - Product dataset
  - Additional mapping and preprocessing files
- These components require **2–4 GB+ RAM**, but Render's free tier offers **only ~512 MB**.
- The Render container repeatedly crashed due to:
  - Out-of-Memory (OOM) errors
  - Backend failing to boot

4. Successful Deployment on Google Cloud Platform (GCP)

4.1 VM Configuration

We deployed the entire application on a single GCP VM instance with:

- **8 GB RAM**
- **~50 GB disk**
- Sufficient compute resources to load:

- ML model
- Embedding vectors
- Dataset
- API services

This configuration resolved all previous memory-related issues.

4.2 Deployment Status

- **Frontend running successfully on GCP VM**
- **Backend running successfully on GCP VM**
- Model, embeddings, and dataset load without failure
- The VM is stopped when not in use to save credits

4.3 Note on Ephemeral IP Address

Since we are using an **ephemeral external IP**, every time we stop and restart the VM:

- The **public IP changes**
- We must update the frontend/backend URLs after each restart

To avoid this, upgrading to a **static external IP** is recommended, but it costs additional credits.

5. Conclusion

- **Frontend Deployment:** Successfully deployed first on Render, and now fully running on the GCP VM.
- **Backend Deployment:** Render deployment failure due to memory limitations was resolved by migrating to an 8 GB RAM GCP VM.
- **Current Deployment:** Both frontend and backend run smoothly on the GCP VM.
- **Limitation:**Due to limited GCP credits, we stop the VM when not in use. As a result, the public IP is **ephemeral** and changes after each restart.

7. Reproducibility Guide (Local Machine + Deployment Guide)

This section explains how anyone can **set up, run, and deploy the AI Stylist system** on their own computer or server.

It includes environment setup, backend + frontend run instructions, FAISS index generation, and deployment steps.

- **Local Setup (Run on Any Machine):**

Follow these steps to run the entire AI Stylist application on laptop/PC.

1.1. Prerequisites: Install the following:

Component	Version
Python	3.9+
pip	Latest
Virtual environment tool	(venv or conda)
Git	Latest
Streamlit	Installed via requirements.txt

1.2. Clone the Repository

```shell
git clone https://github.com/maddy-git/group-5-DS-AI-Lab-Project.git
cd group-5-DS-AI-Lab-Project
```

1.3. Create & Activate Virtual Environment

```shell
python3 -m venv .venv
source .venv/bin/activate       # macOS / Linux
.\.venv\Scripts\activate        # Windows
```

1.4. Install Requirements

File Name: requirements.txt

```python
flask==3.1.2
requests==2025.11.3
streamlit==1.51.0
PyMuPDF==1.26.6
sentence-transformers==5.1.2
scikit-learn==1.7.2
groq==0.34.1
faiss-cpu==1.13.0
numpy==2.3.5
pandas==2.3.3
chromadb==1.3.4
langchain-core==1.0.5
langchain-community==0.4.1
langchain-google-genai==3.0.3
langchain-groq==1.0.1
langchain-classic==1.0.0
pydantic==2.12.4
gunicorn==23.0.0
openai==2.8.1
```

```shell
pip install -r requirements.txt
```

1.5. Prepare the Dataset

- You must download the H&M dataset from Kaggle:https://www.kaggle.com/competitions/h-and-m-personalized-fashion-recommendations/

  Place the following files inside:

```
None
milestones/milestone-2/
```

Required files:

- articles.csv
- customers.csv
- transactions.csv
- sample_images/

1.6. Generate Embeddings + FAISS Index: Only **one-time setup** required.

Run:

```
Shell
cd code
python embed_policy.py
python product_finder_reranker.py   # if used for building product index
python adding_context.py
```

These scripts will:

- Extract text descriptions
- Generate embeddings
- Build FAISS index

- Save them inside /code/content/

This is required for the RAG pipeline to work.

1.7. Run the Backend API

From the code folder:

```
Shell
python app.py
```

Backend will start at:

```
None
http://localhost:8000
```

1.8. Run the Frontend (Streamlit UI)

Open another terminal:

```
Shell
cd code
streamlit run chat_ui.py
```

Streamlit UI will start at:

```
None
http://localhost:8501
```

1.9. Login / Logout

```
None
cust_id:{customer_id}
```

```
None
logout
```

The system now works fully on local machines.

2. Deployment Guide (Cloud Server / GCP / AWS / Azure / Render VM)

This explains how to deploy the application on a VM or cloud environment.

2.1. Choose a Server (Recommended Specs)

Component	Minimum	Recommended
RAM	4 GB	8 GB (for embeddings + model)
Disk	20 GB	50+ GB
CPU	2 cores	4 cores

GCP VM **e2-standard-2 or e2-standard-4** works perfectly.

2.2. Install System Dependencies

SSH into your VM:

```
Shell
sudo apt update
sudo apt install python3 python3-pip python3-venv -y
```

2.3. Clone Repo & Setup Environment

```Shell
git clone https://github.com/maddy-git/group-5-DS-AI-Lab-Project.git
cd group-5-DS-AI-Lab-Project

python3 -m venv .venv
source .venv/bin/activate
pip install -r requirements.txt
```

2.4. Upload Dataset to VM

Upload the files to:

```None
milestones/milestone-2/
```

Use any method:

- SCP (Linux/Mac)
- WinSCP (Windows)
- Google Cloud Upload
- Git LFS

2.5. Build Embeddings on VM

```Shell
cd code
python embed_policy.py
python product_finder_reranker.py
python adding_context.py
```

2.6. Run Backend (Expose Public IP)

```
Shell
python app.py --host 0.0.0.0 --port 8000
```

Allow firewall rule for port 8000.

2.7. Run Frontend (Streamlit on Same VM)

```
Shell
streamlit run chat_ui.py --server.address=0.0.0.0 --server.port=8501
```

- Allow firewall rule for port 8501.

2.8. Connect Frontend ↔ Backend

Update your frontend config if needed:

Inside:

```
None
code/constants.py
```

Set your VM's public IP:

```
None
BACKEND_URL = "http://<your-external-ip>:8000"
```

Restart Streamlit.

2.9. Optional: Use a Static External IP

To avoid changing URLs after VM restart, assign a static IP.

Note: This may cost some cloud credits.

2.10. Keep Services Running (Production Mode)

Use **tmux** or **screen** to keep backend & frontend running after logout.

```
Shell
sudo apt install tmux -y
tmux new -s backend
python app.py
```

Then:

```
Shell
tmux new -s frontend
streamlit run chat_ui.py --server.address=0.0.0.0 --server.port=8501
```

- **Final Summary**
  - Run on any local machine
  - Reproduced end-to-end using embeddings + FAISS
  - Deployed on any cloud VM
  - Accessed publicly as full-stack AI Stylist application

8. References and Appendix

- References drawn from included milestone documents, published papers such as NeuroStylist, Conditional Similarity Networks, and RAGAS documentation.
- Appendix includes sample dataset schemas, scoring sheets, flowcharts, and hyperparameter tuning logs as provided in milestone deliverables.
- **Embedding only the Text Columns:** https://drive.google.com/file/d/1_Q_KcneZ4UML5qXEPNW0D_voho66p9Zs/view?usp=drive_link

- **Embedding the images using Clip:** https://drive.google.com/file/d/12PrSB1A-Su0MROaUSxf1Zslsz66Qdne1/view?usp=sharing
- **Context RAG Colab notebook:** https://colab.research.google.com/drive/1tEEGsEeRPZE7PLtQBnxtFswGLAR656TO?usp=sharing
- **Policy RAG's scoring:** https://docs.google.com/spreadsheets/d/11B68WnwN7omTxreHqbl2SNMprubBCitPI5THU__ij6c/edit?usp=sharing
- **RAGAS evaluation data:** https://docs.google.com/spreadsheets/d/1Vb-UtJMY08MMfZTMcXI8ItL3WcVWtVzAcA7Nrsk1_io/edit?usp=sharing
- **EDA on Product Catalog:** https://drive.google.com/file/d/1_Q_KcneZ4UML5qXEPNW0D_voho66p9Zs/view?usp=sharing
- **EDA On Catalog Data (Product Rag)**
- Link to EDA notebook: https://colab.research.google.com/drive/1vLbQXN8NOBWynEf4U16hXTec5v9bhosz?authuser=1#scrollTo=416491f2-c891-4763-ab10-67d7a2105915
- **Demo video link:**

  group5_demo.mp4